

Axentrio — App Review & OAuth Verification

Submission Packet (Runbook)

Platform: Axentrio chatbot (api.axentrio.com / app.axentrio.com) **Tracks:** A = Meta/Facebook App Review → Live + Advanced Access · B = Google OAuth verification (out of Testing) · B-Appendix = Microsoft/Outlook publisher verification **Last reviewed:** 2026-06-18

Purpose. This is a fill-in form. Follow it top to bottom to (A) take the Axentrio Meta app ([154869899932589](#)) from Development mode into **Live** with **Advanced Access** so customers can connect *their own* Facebook Pages and Instagram professional accounts, and (B) take the shared Google OAuth app out of **Testing** into **verified Production** so external tenants can connect *their own* Google Calendars without the "Google hasn't verified this app" warning and without 7-day refresh-token death. Every field is grounded in the audited codebase; anything unknown is an explicit (where to find it) blank — **do not invent IDs, scopes, or URLs.**

1. Status tracker

Item	Owner	Status (□/☑)	Date
TRACK A — META			
A0. Business Verification complete	ACHRAF LAMRANI MAKHLOUFI	☑	2026-05-26
A1. App config (redirect URIs, webhook, privacy/data-deletion) verified live	_____	□	
A2. ≥1 successful API call per permission made (within 30 days of submit)	_____	□	
A3. Screenshot recorded (1080p, full flow)	_____	□	
A4. Per-permission justifications + tester access entered	_____	□	
A5. App Review submitted	_____	□	
A6. All permissions show Advanced Access	_____	□	
A7. App Mode flipped → Live	_____	□	
TRACK B — GOOGLE			
B1. Consent screen (Branding) configured	_____	□	
B2. Domain verified in Search Console	_____	□	
B3. Scope tags confirmed in Data access page (Sensitive vs Restricted)	_____	□	
B4. Demo video (unlisted YouTube) recorded	_____	□	
B5. Publishing status → Production → Submit for verification	_____	□	
B6. Verification approved (warning gone, 7-day expiry lifted)	_____	□	
TRACK B-APPENDIX — MICROSOFT			
C1. CPP / Partner One ID + publisher domain set	_____	□	
C2. Publisher Verification submitted	_____	□	

Item	Owner	Status (☐/☑)	Date
C3. Blue "verified" badge live on consent prompt	_____	☐	

2. What this unblocks

- **Live customer Meta channels.** Customers (other businesses, with no role on our app) can connect their own Facebook Pages and Instagram professional accounts — today only the Axentrio-owned Page works because every messaging permission is at Standard Access only.
- **The #8 HUMAN_AGENT outside-24h delivery.** Granting the **Human Agent** feature lets human-handoff replies send via `messaging_type=MESSAGE_TAG, tag=HUMAN_AGENT` after the 24h standard window closes; today those sends 400 until the feature is granted (`messenger-transport.ts:84-85` , `instagram-transport.ts:67-68` , gated at `outbound-router.ts:120`).
- **External-tenant Google calendars.** External tenants can connect their own Google Calendar; today the shared OAuth app is in Testing mode with a test-user allowlist, so non-allowlisted accounts are blocked.
- **Removal of the unverified-app warning + lifting the 7-day refresh-token expiry.** Verification removes the "Google hasn't verified this app" interstitial and the ~100-user cap, and stops Testing-mode refresh tokens from expiring 7 days after consent.

3. Prerequisites (do these first — shared by both tracks)

- ☐ Privacy Policy URL live & public: <https://app.axentrio.com/privacy> (route confirmed in `portal/src/App.tsx:382`)
- ☐ Terms of Service URL live: <https://app.axentrio.com/terms> (`App.tsx:383`)
- ☐ Data Deletion (Instructions) URL live: <https://app.axentrio.com/data-deletion> (`App.tsx:384`)
- ☐ Support / contact mailboxes resolve: support@axentrio.com , privacy@axentrio.com
- ☐ Business legal documents on hand (Certificate/Articles of Incorporation, business license, or tax/VAT cert; address+phone proof: utility bill ≤90 days or bank statement) — *Track A already verified; keep for re-use / Microsoft.*
- ☐ **Domain `axentrio.com` verified in Google Search Console** (Owner/Editor) — *required for Track B before submission*
- ☐ App logo/icon ready: **1024×1024**, no Meta/Google trademarks (Meta); square logo for Google consent screen
- ☐ App category accurately set (Meta: Business/Messaging)
- ☐ A clean public homepage that *describes the product* (not just a login) and links the privacy policy — required for Google
- ☐ A dedicated **test Facebook account** added as a **Tester** (App Roles → Roles → Testers) with known credentials — **never submit personal Meta credentials**
- ☐ A throwaway external **Google account** (not on the Testing allowlist) to validate after Track B

TRACK A — META APP REVIEW

A(a) Business Verification steps

Status: ☑ **DONE** (ACHRAF LAMRANI MAKHLOUFI, 2026-05-26). This is a hard gate for Advanced Access — keep the verified-business → app link intact. Steps below are for reference / re-verification only.

1. ☐ A **Business Admin** opens Business Manager → Business Settings → **Security Centre** → **Start Verification**.
2. ☐ Confirm organization details (legal name, address, phone) — must match documents exactly.
3. ☐ Upload supporting documents (incorporation/license/VAT cert; utility bill or bank statement ≤90 days; uncropped, readable, seals visible).
4. ☐ Choose a confirmation-code method (phone/email) **or** verify the domain to skip it.
5. ☐ Enter the confirmation code. Decision typically **1–3 business days**.
6. ☐ Ensure 2FA is enabled on the submitting account.

A(b) App config form

Enter / confirm these in **App Dashboard** → **Settings** and **Messenger / Instagram** → **Webhooks / Settings**.

Code uses **classic Facebook Login** (no `config_id` — grep-confirmed), Graph **v25.0**, and **Strict Mode ON** for redirect URIs so `META_OAUTH_REDIRECT_URI` must *exactly* match a whitelisted URI.

Field	Value to enter
App ID	<code>1548698999932589</code> (env <code>META_APP_ID</code>)
App Secret	_____ (App Dashboard → Settings → Basic → App Secret; env <code>META_APP_SECRET</code>)
Business Manager ID	<code>2010737469572288</code>
App type	Business
App category	_____ (Business / Messaging — Settings → Basic → Category)
App domains	<code>axentrio.com</code> , <code>api.axentrio.com</code> , <code>app.axentrio.com</code>
Privacy Policy URL	<code>https://app.axentrio.com/privacy</code>
Terms of Service URL	<code>https://app.axentrio.com/terms</code>
Data Deletion (Instructions) URL	<code>https://app.axentrio.com/data-deletion</code>
App icon	_____ (1024×1024 upload)
Graph API version	<code>v25.0</code> (single source of truth <code>graph-api.ts:10</code>)
OAuth redirect URI (primary)	<code>https://api.axentrio.com/api/v1/channels/meta/oauth/callback</code> (env <code>META_OAUTH_REDIRECT_URI</code>)
OAuth redirect URI (legacy — keep whitelisted)	<code>https://chatbot-api-production-d7d4.up.railway.app/api/v1/channels/meta/oauth/callback</code>
Login variant	Classic Facebook Login (no <code>config_id</code>); <code>auth_type=rerequest</code> to force re-consent
OAuth dialog	<code>https://www.facebook.com/v25.0/dialog/oauth</code>
Webhook callback URL (Page/Messenger+IG)	_____/api/v1/channels/meta/webhook — ⚠ prod currently points at <code>https://chatbot-api-production-d7d4.up.railway.app/api/v1/channels/meta/webhook</code> , NOT <code>api.axentrio.com</code> . Confirm/repoint before submit.
Webhook verify token	value of env <code>META_VERIFY_TOKEN</code> — _____ (App Dashboard; webhook fails closed if unset)
Webhook HMAC	<code>X-Hub-Signature-256</code> = SHA-256 HMAC of raw body using <code>META_APP_SECRET</code> (<code>graph-webhook.ts:34–41</code>)
Webhook fields — Page level	<code>messages</code> , <code>messaging_postbacks</code> , <code>messaging_optins</code> , <code>message_deliveries</code> , <code>message_reads</code> , <code>messaging_referrals</code> (<code>setup.service.ts:67</code>)

Field	Value to enter
Webhook fields — Instagram level	<code>messages</code> , <code>messaging_postbacks</code> , <code>message_reactions</code> (<code>setup.service.ts:161</code>)
OAuth state JWT secret	env <code>META_OAUTH_JWT_SECRET</code> (32+ chars in prod) — not entered in dashboard, infra only

A(c) Permissions to request

Request **exactly these six** at Advanced Access, plus the **Human Agent feature** (only because we have a real human-handoff path that sends late replies). **Do NOT add** `pages_read_engagement` or `public_profile` — they are not requested anywhere in `api/src` (grep-confirmed). Meta may surface dependency permissions (`pages_show_list` , etc.) automatically; accept those it lists.

Permission / Feature	Why (our use)	Code evidence
<code>pages_messaging</code>	Send/receive Messenger messages on connected Pages	<code>messenger-transport.ts:43</code> (<code>/ {pageId}/messages</code>); inbound field <code>messages</code> <code>setup.service.ts:67</code>
<code>pages_manage_metadata</code>	Subscribe the Page to our webhooks to receive inbound events	<code>setup.service.ts:61</code> (<code>/ {pageId}/subscribed_apps</code>); disconnect <code>disconnect.service.ts:24</code>
<code>pages_show_list</code>	List the Pages a person manages so they can pick which to connect	<code>oauth.service.ts:115</code> (<code>/me/accounts</code>); scope <code>oauth.service.ts:16</code>
<code>business_management</code>	Read Business-Portfolio-owned Pages + linked IG during connect	<code>oauth.service.ts:128</code> (<code>/me/businesses</code>) , <code>:135</code> (<code>/ {biz.id}/owned_pages</code>)
<code>instagram_basic</code>	Read connected IG account id/username/profile pic for display + DM routing	<code>oauth.service.ts:114</code> & <code>:174-179</code> ; <code>profile.service.ts:27-29</code>
<code>instagram_manage_messages</code>	Send/receive Instagram Direct messages on connected IG accounts	<code>instagram-transport.ts:41</code> (<code>/ {igBusinessId}/messages</code>); IG subscribed_fields <code>setup.service.ts:161</code>
Human Agent (feature)	Send human-staff replies via <code>HUMAN_AGENT</code> tag <i>after</i> the 24h window	<code>messenger-transport.ts:84-85</code> , <code>instagram-transport.ts:67-68</code> ; gate <code>outbound-router.ts:120</code> ; set on handoff <code>socket.handler.ts:590</code>

Ready-to-paste justifications (first person, for the reviewer)

Intro (paste at the top of each):

Axentrio is a business-to-business platform that lets a business connect its own Facebook Page and Instagram professional account to an AI assistant that answers customer messages, captures leads, and schedules bookings. The business signs in to our dashboard at `app.axentrio.com`, connects its Page/account via Facebook Login, and from then on we receive inbound messages by webhook and reply on the business's behalf. We only access data for Pages/accounts the business explicitly connects, and only to provide the messaging features they enable.

`pages_messaging`

We use this permission to send and receive Messenger conversations on behalf of the Pages our customers connect. When an end user messages a connected Page, we receive the message through the `messages`

webhook field and send the AI assistant's reply back via the Send API at `/{"page-id"}/messages` , inside the standard 24-hour messaging window. Without it the core product — answering Messenger customers automatically — cannot function. We never message users who have not first messaged the connected Page.

`pages_manage_metadata`

We use this permission solely to subscribe each connected Page to our app's webhook (`POST /{"page-id"}/subscribed_apps`) so that inbound `messages` and `messaging_postbacks` events are delivered to us, and to unsubscribe on disconnect. Without it we cannot register the Page's webhook subscription and would never receive inbound messages. We do not edit Page content with it.

`pages_show_list`

We use this permission to call `GET /me/accounts` and show the customer the list of Pages they manage, so they can choose which single Page to connect to their AI assistant. It is the picker step of the connect flow; we store only the Page the customer selects.

`business_management`

Many of our customers manage their Pages through a Business Portfolio rather than as a personal Page. We use this permission to read their Business-Manager-owned Pages (`GET /me/businesses` then `/{"business-id"}/owned_pages`) and the Instagram accounts linked to them, so business-managed Pages appear in the connect picker. We read these assets only to let the customer select what to connect.

`instagram_basic`

We use this permission to read the connected Instagram professional account's `id` , `username` , and `profile_picture_url` so we can display the connected account in the dashboard and route inbound Direct messages to the correct account. We request only these profile fields and only for accounts the customer connects.

`instagram_manage_messages`

We use this permission to send and receive Instagram Direct messages for the Instagram professional accounts our customers connect, delivering the same AI-assistant experience as Messenger. Inbound DMs arrive via the IG-level `messages` subscription and replies are sent through the Instagram messaging Send API. Without it we cannot support Instagram as a channel at all.

Human Agent (feature)

We request the Human Agent feature because our platform includes a human-handoff inbox: when the AI escalates a conversation, a human staff member can reply. If that human reply happens after the 24-hour standard window has closed, we send it using `messaging_type=MESSAGE_TAG` with `tag=HUMAN_AGENT` . We use this tag exclusively for genuine human-agent replies (set on the human-handoff code path), never for automated/bot messages, and only when the standard window has elapsed.

A(d) Screencast / demo-video SCRIPT

One ~60–90s take, **1080p+**, English UI, captions/tooltips, minimal audio. Reviewers do not explore the app — show the **complete flow**; jumping straight to the permission-in-use is the #1 rejection cause. Use the **Tester** account, not personal credentials.

1. ☐ Open `https://app.axentrio.com` , already signed in. Show the dashboard, click **Channels** in the left nav.
2. ☐ Click **Connect** on **Facebook Messenger** → the Facebook Login dialog appears.

3. ☐ On screen, show the **permissions being granted** in the consent dialog (the scopes listed: pages_messaging, pages_manage_metadata, pages_show_list, business_management, instagram_basic, instagram_manage_messages).
4. ☐ Show the **Page picker** (`/me/accounts` result) → select the test Page → land back on a **Connected** state.
5. ☐ (If demoing IG) repeat Connect on **Instagram**, grant, select the linked IG professional account → **Connected**.
6. ☐ Switch to a phone / second account → open Messenger to the connected Page → send: *"Hi, do you have availability this week?"*
7. ☐ Show the **AI reply** arriving automatically in the conversation (a real booking exchange is ideal).
8. ☐ (Human Agent shot) From the Axentrio inbox, trigger **human handoff** and send a staff reply; show it delivered in the thread. (Use a thread where the last user message is recent so the send succeeds.)
9. ☐ End on the full conversation thread showing the complete exchange.

A(e) Submit + go-Live checklist

- ☐ In Dev mode, make **≥1 successful real API call per permission** (Messenger send/receive, IG send/receive, `/me/accounts`, `/me/businesses`) **within 30 days of submitting**.
- ☐ Confirm all App Settings URLs resolve publicly (privacy, terms, data-deletion) and the app icon is set.
- ☐ Confirm webhook callback URL + verify token are correct (**re-check the Railway-vs-api.axentrio.com host note above**).
- ☐ In **App Review** → **Permissions and Features**, select the 6 permissions + Human Agent feature.
- ☐ Answer the **data-handling questions**; paste per-permission justifications (A(c)).
- ☐ Upload the **screencast**; provide **Tester** credentials and the **Steps to reproduce** (from `docs/meta-app-review.md:76-88`).
- ☐ **Submit**. SLA ~1 week (~5 business days; longer on resubmission).
- ☐ On approval, confirm each permission shows **Advanced Access**.
- ☐ Flip **App Mode** → **Live** (top of App Dashboard). Customers can now connect their own Pages/IG accounts.

TRACK B — GOOGLE OAUTH VERIFICATION

The app books appointments on tenants' Google Calendars → needs **write** access (sensitive scopes) → requires brand + sensitive-scope verification. **One shared** `GOOGLE_CLIENT_ID` / `GOOGLE_REDIRECT_URI` **serves all tenants**, so everyone funnels through the single app that is currently in Testing. The narrow `calendar.events` scope (not full `calendar`) is what keeps this in the lighter sensitive-only lane and **off CASA** — see B(b).

B(a) OAuth consent screen form (Google Auth Platform → Branding / Data access / Audience)

Field	Value
App name	<code>Axentrio</code>
User support email	<code>support@axentrio.com</code>
App logo	<code>_____</code> (square logo upload – note: changing the logo later re-triggers verification)
Application home page	<code>https://app.axentrio.com</code> (must be public, describe the product, link the privacy policy)
Privacy policy URL	<code>https://app.axentrio.com/privacy</code>

Field	Value
Terms of service URL	<code>https://app.axentrio.com/terms</code>
Authorized domain(s)	<code>axentrio.com</code> (must be Search-Console-verified)
Developer contact email	_____ (kept current – Trust & Safety uses it)
Client ID	env <code>GOOGLE_CLIENT_ID</code> – _____ (Cloud Console → Credentials)
Authorized redirect URI (primary)	<code>https://api.axentrio.com/api/v1/integrations/google/callback</code> (env <code>GOOGLE_REDIRECT_URI</code> ; mounted <code>server.ts:279</code>)
Authorized redirect URI (legacy fallback, if present)	_____ (Railway-style URI may exist in console as harmless fallback; runtime uses the env one only)
Publishing status	Testing → set to Production, then Submit for verification
User type	External

B(b) Scopes table

Source of truth: `SCOPES` array in `google-calendar.service.ts:25-30` (these four — nothing else). `auth_type` : `access_type=offline` + `prompt=consent` + `include_granted_scopes=true` to obtain/refresh refresh tokens.

Scope URL	Classification	Notes
<code>openid</code>	Non-sensitive	Sign-in identity
<code>email</code>	Non-sensitive	Read tenant email + <code>email_verified</code> from <code>id_token</code>
<code>https://www.googleapis.com/auth/calendar.events</code>	Sensitive	Create/read/update/delete <i>events only</i> — used for booking, reschedule, cancel, Meet link
<code>https://www.googleapis.com/auth/calendar.calendarlist.readonly</code>	Sensitive	List writable calendars for the calendar picker (<code>minAccessRole=writer</code>)

CASA question — answered: We request **only** `calendar.events` (event-level), **not** the broad `https://www.googleapis.com/auth/calendar` (full read/write/delete of all calendars) and **not** `calendar.freebusy` — busy times are computed via `Events.list` specifically to avoid the freebusy scope. Both requested calendar scopes are **Sensitive**, not Restricted, so the path is **brand + sensitive-scope verification only (~days)**, with **no CASA security assessment** in the typical case. **Switching away from the broad `calendar` scope is the single biggest lever that avoids CASA** — keep it. **Operator must still confirm on-screen:** in Cloud Console → **Data access**, read the tag Google shows next to each calendar scope for *this* project; that on-screen tag (Sensitive vs Restricted), not the docs, governs the burden. If either shows **Restricted**, budget for CASA (weeks + ~\$540–\$1,500/yr, annual re-validation).

Ready-to-paste scope justifications

`calendar.events`

Axentrio is an AI booking assistant. When an end user books, reschedules, or cancels an appointment in chat, we create, update, or delete the corresponding event on the tenant's connected Google Calendar (with a Google Meet link). We use `calendar.events` — the least-privilege scope — because read-only access is insufficient (we

must write events) and the full `calendar` scope is unnecessary (we never need to create/delete entire calendars or view calendars we did not create events on).

`calendar.calendarlist.readonly`

After a tenant connects Google, we show them a picker of the calendars they can write to so they can choose which calendar bookings land on. We call `CalendarList.list` with `minAccessRole=writer` to list only writable calendars. This read-only scope is the minimum needed to populate that picker; without it the tenant cannot select a target calendar.

B(c) Domain verification steps

1. ☐ Sign in to **Google Search Console** with an account that is Owner/Editor of the project (or add it).
2. ☐ Add property `axentrio.com` (Domain property recommended) and complete **DNS TXT** verification (NS1 DNS for this domain).
3. ☐ Confirm the verified domain appears under **Authorized domains** on the consent screen.
4. ☐ Confirm homepage `https://app.axentrio.com` and `https://app.axentrio.com/privacy` are publicly reachable (not login-gated) and on the verified domain.

B(d) Demo-video SCRIPT (unlisted YouTube, English)

1. ☐ Start in the Axentrio dashboard → **Settings / Integrations** → **Connect Google Calendar**.
2. ☐ Click connect → the **Google OAuth consent screen** appears. **Hold on it** so the reviewer can read the **app name "Axentrio"** and the **exact scopes** (`calendar.events` + `calendar.calendarlist.readonly`).
3. ☐ Show the **browser address bar with the OAuth client ID** visible in the URL (proves it's our client) during the consent step.
4. ☐ Grant consent → land back in Axentrio showing the **calendar picker** (the `calendarlist.readonly` scope in use) → select a calendar.
5. ☐ In chat, **book an appointment** → show the event created on the Google Calendar (the `calendar.events` scope in use), including the Meet link.
6. ☐ **Reschedule** then **cancel** the same booking → show the event updated/removed on the calendar (demonstrates create/update/delete).
7. ☐ End on the Google Calendar reflecting the final state.

B(e) Submit + monitor checklist

- ☐ Confirm scopes in Data access = exactly the four above (no `calendar`, no `freebusy`).
- ☐ Confirm privacy policy discloses Google user-data access/use/storage/sharing + **Limited Use** compliance.
- ☐ Confirm domain verified, homepage public and descriptive, logo/name/support email set.
- ☐ Set publishing status to **Production** → **Publish App**, then **Prepare for Verification / Submit**.
- ☐ Paste scope justifications; attach the unlisted YouTube link.
- ☐ **Submit**. SLA: brand ~2–3 business days; sensitive-scope "up to 10 days" (longer with back-and-forth).
- ☐ Watch the developer-contact inbox — Trust & Safety emails for clarifications; **respond fast**.
- ☐ On approval: confirm the unverified warning is gone and Testing-mode test-user allowlist no longer applies.

TRACK B — APPENDIX: Microsoft / Outlook

The codebase has a live Outlook integration (`outlook-calendar.service.ts` , `outlook-events.service.ts` , mounted `server.ts:281`). Per `docs/booking-records-outlook-p6.md` the Azure app is registered **multitenant + personal** ("Accounts in any organizational directory and personal Microsoft accounts"). Since Nov 2020, risk-based step-up consent blocks users from consenting to most newly-registered **multitenant** apps unless the publisher is verified — so **Publisher Verification is effectively mandatory**. It is free and can be done in minutes once prerequisites are met. **No CASA-equivalent** for basic Graph calendar access.

App config

Field	Value
Client ID	env <code>MICROSOFT_CLIENT_ID</code> — _____ (Entra → App registrations)
Client secret	env <code>MICROSOFT_CLIENT_SECRET</code> — _____
Redirect URI	<code>https://api.axentrio.com/api/v1/integrations/outlook/callback</code> (env <code>MICROSOFT_REDIRECT_URI</code>)
Supported account types	Multitenant + personal Microsoft accounts
Publisher domain	<code>axentrio.com</code> (must be set; cannot be <code>*.onmicrosoft.com</code>)
Authority	<code>https://login.microsoftonline.com/common/oauth2/v2.0/{authorize,token}</code>

Scopes (delegated; `outlook-calendar.service.ts`): `offline_access` , `openid` , `email` , `User.Read` , `Calendars.ReadWrite` .

`Calendars.ReadWrite` (delegated) creates/updates/deletes events on the signed-in user's own calendar and generally does **not** require admin consent — though some org admins can require admin consent for multitenant apps. `offline_access` yields the rotated refresh token (persisted under a per-bot pg advisory lock).

Publisher Verification checklist

- ☐ C1. Have a verified **Microsoft AI Cloud Partner Program (CPP)** account with a **Partner One ID** set as the org's Partner Global Account (PGA).
- ☐ C2. App registered with a **work/school account** (not personal), in a tenant associated with the PGA, with the **publisher domain set** (`axentrio.com` , not `*.onmicrosoft.com`).
- ☐ C3. Verification email domain matches the publisher domain (or a DNS-verified custom domain).
- ☐ C4. Initiator has **Application Administrator / Cloud Application Administrator** in Entra **and** Partner/Account Admin in Partner Center, with **MFA**.
- ☐ C5. Submit Publisher Verification → confirm the **blue "verified" badge** on the consent prompt.

7. After approval — verify it worked

Meta:

- ☐ From a Facebook account with **no role** on our app, connect a **Page we don't own** end-to-end; confirm it reaches **Connected**.
- ☐ Connect an **Instagram professional account** (non-owned) and confirm inbound DMs route + AI replies send.
- ☐ Send an inbound message to the new Page, confirm the AI reply arrives within the 24h window (RESPONSE type).
- ☐ **Human Agent**: force a thread where the last user message is **>24h old**, trigger human handoff, send a staff reply, and confirm it delivers (no 400) via `tag=HUMAN_AGENT` .
- ☐ Confirm webhook events are arriving at the **production** callback host (re-check the Railway-vs-api.axentrio.com note).

Google:

- ☐ From an **external Google account that was never on the Testing allowlist**, connect a calendar — confirm **no "Google hasn't verified this app" warning**.
- ☐ Book → reschedule → cancel an event; confirm it appears/updates/deletes on that external calendar with a Meet link.
- ☐ Confirm the **refresh token persists >7 days** (re-test a booking ≥ 8 days after consent without re-auth — proves Testing-mode 7-day expiry is gone).

Microsoft:

- ☐ From an external Microsoft account, connect Outlook; confirm the **verified-publisher badge** shows on consent and a booking creates an event with a Teams link.

8. Risks / gotchas

- **Standard-Access false positive (Meta)**. The app *works on our own/test Pages* even before approval — do not mistake that for working on customer Pages. Only Advanced Access enables non-owned Pages.
- **Webhook host mismatch (Meta)**. Prod Messenger webhook callback still points at `chatbot-api-production-d7d4.up.railway.app/...`, not `api.axentrio.com`. Verify before submit or inbound events may not reach the reviewed host.
- **Strict Mode redirect (Meta)**. `META_OAUTH_REDIRECT_URI` must *exactly* match a whitelisted URI; both the `api.axentrio.com` and legacy Railway URLs are whitelisted — keep both.
- **Human Agent misuse = fast ban (Meta)**. The `HUMAN_AGENT` tag must only be used for genuine human replies; tagging bot/automated messages is a policy violation. Our code already gates it to the human-handoff path outside the window — keep it that way and demonstrate a *human* sending it.
- **Top Meta rejection causes**: requesting unused permissions (we deliberately exclude `pages_read_engagement` / `public_profile`); screencast skipping login→grant→select; permission demoed but feature not actually built; unreachable privacy/data-deletion URLs; personal (not Tester) credentials; missing the ≥ 1 successful API call per permission within 30 days.
- **Testing-mode 7-day refresh-token death (Google)**. Until verified, Testing refresh tokens expire 7 days from consent — booking silently breaks for tenants after a week. This is the core reason to verify, not just convenience.
- **CASA risk (Google)**. If Cloud Console's Data access page tags either calendar scope **Restricted** for this project, CASA applies (weeks + ~\$540–\$1,500/yr, annual; Tier-2 self-scan is being phased out in favor of lab-validated scans). Staying on `calendar.events` is the mitigation — never add full `calendar`.
- **Single shared Google client (Google)**. All tenants use one `GOOGLE_CLIENT_ID` /redirect; one verification covers everyone, but a logo change after approval re-triggers verification — finalize branding first.
- **App-level gate is separate**. Even after Google verification, `requireFeature(tenantId, 'calendarSync')` (Pro+ entitlement) gates connect/callback/list before OAuth — non-entitled tenants still can't connect; that is intended product behavior, not a verification failure.
- **Redirect-URI propagation (both)**. New/edited redirect URIs and consent-screen/branding changes can take time to propagate; allow a propagation window before testing the live flow.
- **Microsoft multitenant consent block**. Without Publisher Verification, step-up consent blocks external users on the multitenant app — verify the publisher before external launch.